

Long-Short Term Memory Networks Hand-Writing Generator

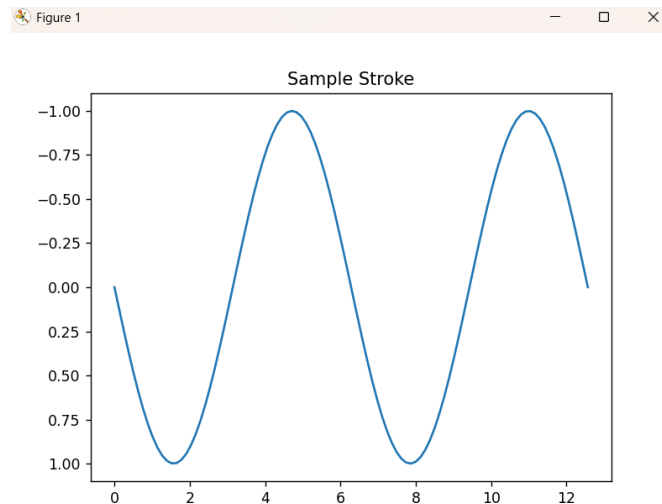
Anirudh A C | BL.EN.U4RAE23003

[Click Here to Access the GitHub Repo for Code](#)

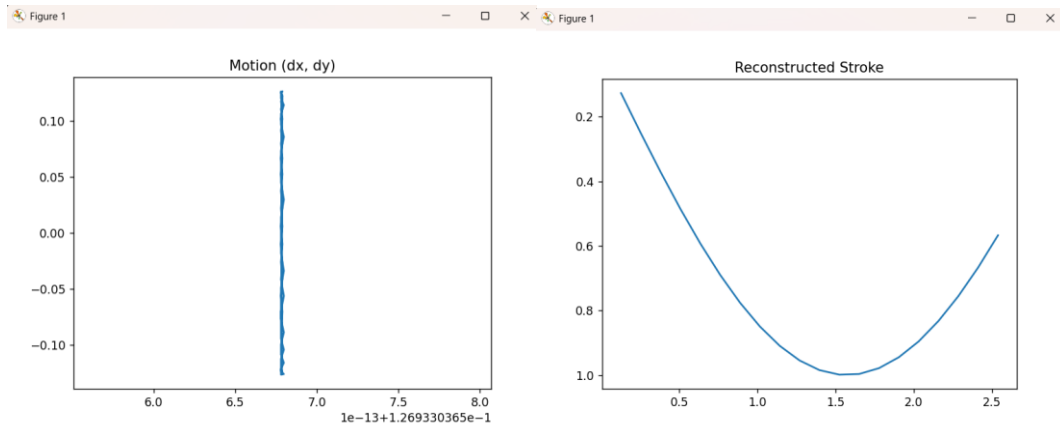
I. Introduction

Handwriting can be represented as a sequence of pen positions over time, making it a time-series problem. Each point in the sequence depends on previous points, which makes sequential models suitable for learning handwriting dynamics. Traditional approaches focus on image-based handwriting recognition, but this project models handwriting as a trajectory generation problem.

The objective is to train a model that learns how a pen moves and generates new handwriting trajectories. This is achieved using Long Short-Term Memory (LSTM) networks, which are capable of modeling temporal dependencies in sequential data.



The system takes real handwriting data, processes it into coordinate sequences, and trains models to predict the next point in a sequence. Generated sequences are then combined to form complete words and visualized as handwriting.



II. Dataset

The dataset used is the UJI Pen Characters dataset, which contains online handwriting data captured as sequences of pen coordinates.

Each sample consists of:

- multiple strokes
- each stroke containing (x, y) points
- pen-up and pen-down information

The data is stored in a structured text format with markers such as:

- .SEGMENT → new character
- .PEN_DOWN → start of stroke
- .PEN_UP → end of stroke

Component Description

Character	Label (A–Z)
Stroke	Sequence of points
Coordinates	(x, y) values
Pen State	Stroke boundaries

Preprocessing Steps

1. Extract strokes and labels
2. Flatten strokes into sequences
3. Normalize coordinates:

$$x' = \frac{x - \mu_x}{\sigma_x}, y' = \frac{y - \mu_y}{\sigma_y}$$

4. Create sliding windows of length L

III. Methodology

The system is designed to learn handwriting trajectories using sequence prediction.

Training Process

- Input: sequence of length L
- Output: next sequence shifted by 1

$$X = [(x_1, y_1), \dots, (x_L, y_L)]$$

$$Y = [(x_2, y_2), \dots, (x_{L+1}, y_{L+1})]$$

The model learns to predict the next point in the sequence.

Per-Character Modeling

Instead of a single model, separate models are trained for each character:

- reduces variation
- improves shape consistency

Generation

1. Take initial seed sequence
2. Predict next point
3. Append prediction
4. Repeat for T steps

Word Formation

Words are generated by:

- generating individual letters
- aligning them
- adding spacing

IV. Model Architecture

The model used is a multi-layer LSTM network.

Layer	Description
Input	2 (x, y)
LSTM	128 hidden units
Layers	2
Dropout	0.2
Output	2 (next x, y)

Forward pass:

$$\hat{Y} = \text{Linear}(\text{LSTM}(X))$$

V. Implementation

The system is implemented in Python using PyTorch.

Component	Description
Data Loader	Parses UJI dataset
Preprocessing	Normalization + sequences
Model	LSTM network
Training	MSE loss + Adam optimizer
Inference	Generates handwriting

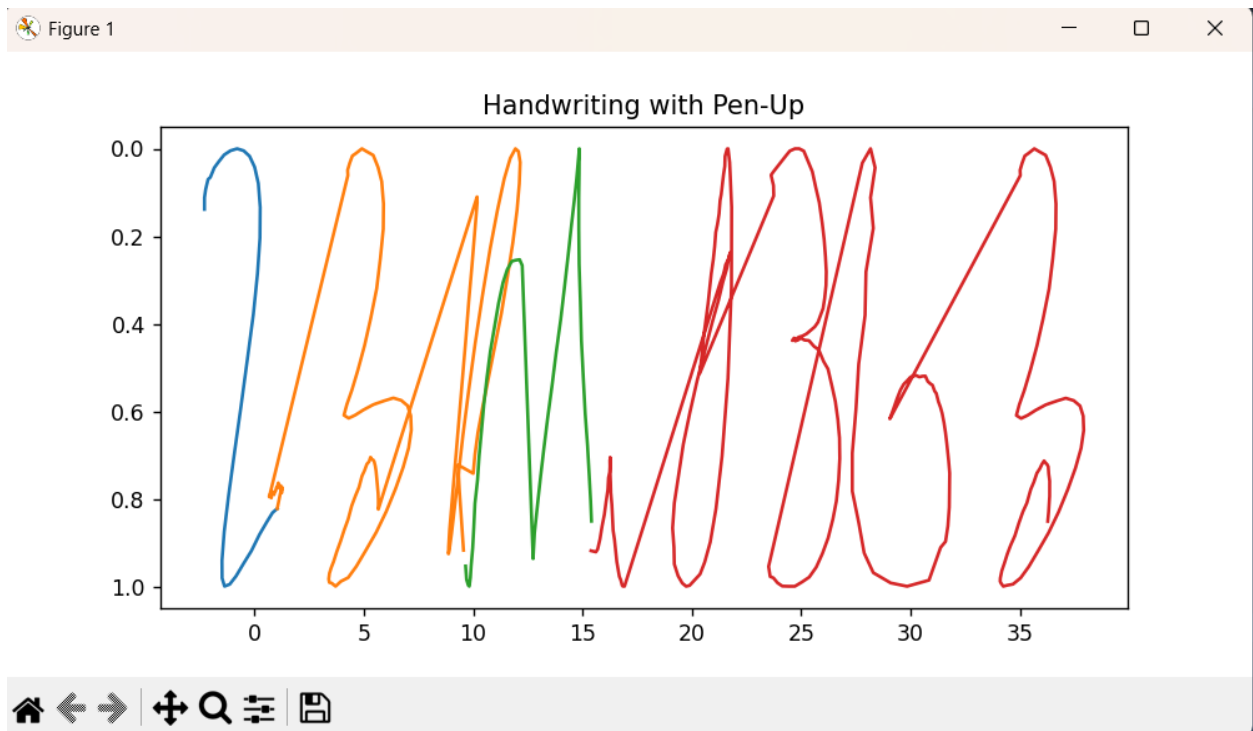
Training Configuration

Parameter	Value
Sequence Length	40
Batch Size	64

Parameter	Value
Epochs	40
Learning Rate	0.001

VI. Results and Limitations

The model is able to generate smooth handwriting trajectories and construct words.



The model tried to write 23RAI313. Since it has no pen-up function, it looks shabby.

Observations

- LSTM captures local motion dynamics
- Generated strokes are smooth
- Per-character models improve consistency
- Pen-up handling improves readability

Limitations

- Model learns local motion, not full shape
- No explicit pen-state prediction
- Drift occurs in long sequences

- Limited dataset size

VII. Conclusion

This project demonstrates that LSTM networks can effectively model handwriting as a sequence of pen movements. While the model captures motion dynamics well, it does not fully learn global character structure.

Future improvements include:

- pen-state prediction
- advanced generative models
- integration with robotic systems